

- Description

commons-math-4.0-beta1-src\commons-math-legacy\src\main\java\org\apache\commons\math4\legacy\analysis\interpolation\LoessInterpolator.java

- Specification

module LoessInterpolator

type

x, y: sequence of real
 n: int
 bandwidth: real
 robustnesslters: int
 accuracy: real
 ex: set of {
 None,
 NullPointerException,
 NoDataException,
 DimensionMismatchException,
 NumberIsTooSmallException,
 NonMonotonicSequenceException,
 NotFiniteNumberException,
 OutOfRangeException,
 NotPositiveException,
 NotStrictlyPositiveException
 }

process Construct(bandwidth: real, robustnesslters: int, accuracy: real) outputs: ex: set

Functional Scenario Form

bandwidth <= 0	&& ex = OutOfRangeException
robustnesslters < 0	&& ex = NotPositiveException
accuracy <= 0	&& ex =
NotStrictlyPositiveException	
bandwidth > 1	&& ex = None

end_process

process Smooth(x, y: sequence of real) outputs: ex: set

Functional Scenario Form

(x = null or y = null)	&& (ex = NullPointerException)
or	
(x = 0 or y = 0)	&& (ex = NoDataException) or
(x ≠ y)	&& (ex =
DimensionMismatchException) or	

```

(|x| < 2)                                && (ex =
NumberIsTooSmallException) or
  (∃ i • i ∈ 0..|x|-1 ∧ (¬finite(x[i]) or ¬finite(y[i])))
                                && (ex =
NotFiniteNumberException) or
  (¬strict_increasing(x))          && (ex =
NonMonotonicSequenceException) or
  (true)                            && (ex = None)
end_process

```

process SmoothWithParams(x, y: sequence of real,
bandwidth: real, robustnesslters: int, accuracy: real) outputs:
ex: set

Functional Scenario Form

```

// Parameter checks first (constructor-equivalent)
  (bandwidth <= 0 or bandwidth > 1)          && (ex =
OutOfRangeException) or
  (robustnesslters < 0)                      && (ex = NotPositiveException)
or
  (accuracy <= 0)                            && (ex =
NotStrictlyPositiveException) or

  // Data checks (same as Smooth)
  (x = null or y = null)                    && (ex = NullArgumentException)
or
  (|x| = 0 or |y| = 0)                      && (ex = NoDataException) or
  (|x| ≠ |y|)                                && (ex =
DimensionMismatchException) or
  (|x| < 2)                                  && (ex =
NumberIsTooSmallException) or
  (∃ i • i ∈ 0..|x|-1 ∧ (¬finite(x[i]) or ¬finite(y[i])))
                                && (ex =
NotFiniteNumberException) or
  (¬strict_increasing(x))                    && (ex =
NonMonotonicSequenceException) or
  (true)                                    && (ex = None)
end_process

```

end_module

- Java Code

```
/*
```

```

* Licensed to the Apache Software Foundation (ASF) under one or more
* contributor license agreements. See the NOTICE file distributed with
* this work for additional information regarding copyright ownership.
* The ASF licenses this file to You under the Apache License, Version
2.0
* (the "License"); you may not use this file except in compliance with
* the License. You may obtain a copy of the License at
*
*     http://www.apache.org/licenses/LICENSE-2.0
*
* Unless required by applicable law or agreed to in writing, software
* distributed under the License is distributed on an "AS IS" BASIS,
* WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or impl
ied.
* See the License for the specific language governing permissions and
* limitations under the License.
*/
package org.apache.commons.math4.legacy.analysis.interpolation;
import java.util.Arrays;
import org.apache.commons.math4.legacy.analysis.polynomials.PolynomialS
plineFunction;
import org.apache.commons.math4.legacy.exception.DimensionMismatchExcep
tion;
import org.apache.commons.math4.legacy.exception.NoDataException;
import org.apache.commons.math4.legacy.exception.NonMonotonicSequenceEx
ception;
import org.apache.commons.math4.legacy.exception.NotFiniteNumberExcepti
on;
import org.apache.commons.math4.legacy.exception.NotPositiveException;
import org.apache.commons.math4.legacy.exception.NumberIsTooSmallExcept
ion;
import org.apache.commons.math4.legacy.exception.OutOfRangeException;
import org.apache.commons.math4.legacy.exception.util.LocalizedFormats;
import org.apache.commons.math4.core.jdkmath.JdkMath;
import org.apache.commons.math4.legacy.core.MathArrays;
import java.util.Random;
import org.apache.commons.math4.legacy.exception.*;
/**
 * Implements the <a href="http://en.wikipedia.org/wiki/Local_regression
n">
 * Local Regression Algorithm</a> (also Loess, Lowess) for interpolatio
n of
 * real univariate functions.
 * <p>

```

```

* For reference, see
* <a href="http://amstat.tandfonline.com/doi/abs/10.1080/01621459.1979
.10481038">
* William S. Cleveland - Robust Locally Weighted Regression and Smooth
ing
* Scatterplots</a></p>
* <p>
* This class implements both the loess method and serves as an interpo
lation
* adapter to it, allowing one to build a spline on the obtained loess
fit.</p>
*
* @since 2.0
*/
public class LoessInterpolator
    implements UnivariateInterpolator {
    /** Default value of the bandwidth parameter. */
    public static final double DEFAULT_BANDWIDTH = 0.3;
    /** Default value of the number of robustness iterations. */
    public static final int DEFAULT_ROBUSTNESS_ITERS = 2;
    /**
     * Default value for accuracy.
     * @since 2.1
     */
    public static final double DEFAULT_ACCURACY = 1e-12;
    /**
     * The bandwidth parameter: when computing the loess fit at
     * a particular point, this fraction of source points closest
     * to the current point is taken into account for computing
     * a least-squares regression.
     * <p>
     * A sensible value is usually 0.25 to 0.5.</p>
     */
    private final double bandwidth;
    /**
     * The number of robustness iterations parameter: this many
     * robustness iterations are done.
     * <p>
     * A sensible value is usually 0 (just the initial fit without any
     * robustness iterations) to 4.</p>
     */
    private final int robustnessIters;
    /**
     * If the median residual at a certain robustness iteration

```

```

    * is less than this amount, no more iterations are done.
    */
    private final double accuracy;
    /**
     * Constructs a new {@link LoessInterpolator}
     * with a bandwidth of {@link #DEFAULT_BANDWIDTH},
     * {@link #DEFAULT_ROBUSTNESS_ITERS} robustness iterations
     * and an accuracy of {@link #DEFAULT_ACCURACY}.
     * See {@link #LoessInterpolator(double, int, double)} for an expla
nation of
     * the parameters.
     */
    public LoessInterpolator() {
        this.bandwidth = DEFAULT_BANDWIDTH;
        this.robustnessIters = DEFAULT_ROBUSTNESS_ITERS;
        this.accuracy = DEFAULT_ACCURACY;
    }
    /**
     * Construct a new {@link LoessInterpolator}
     * with given bandwidth and number of robustness iterations.
     * <p>
     * Calling this constructor is equivalent to calling {link {@link
     * #LoessInterpolator(double, int, double) LoessInterpolator(bandwi
dth,
     * robustnessIters, LoessInterpolator.DEFAULT_ACCURACY)}
     * </p>
     *
     * @param bandwidth when computing the loess fit at
     * a particular point, this fraction of source points closest
     * to the current point is taken into account for computing
     * a least-squares regression.
     * A sensible value is usually 0.25 to 0.5, the default value is
     * {@link #DEFAULT_BANDWIDTH}.
     * @param robustnessIters This many robustness iterations are done.
     * A sensible value is usually 0 (just the initial fit without any
     * robustness iterations) to 4, the default value is
     * {@link #DEFAULT_ROBUSTNESS_ITERS}.
     * @see #LoessInterpolator(double, int, double)
     */
    public LoessInterpolator(double bandwidth, int robustnessIters) {
        this(bandwidth, robustnessIters, DEFAULT_ACCURACY);
    }
    /**
     * Construct a new {@link LoessInterpolator}

```

```

    * with given bandwidth, number of robustness iterations and accuracy.
    *
    * @param bandwidth when computing the loess fit at
    * a particular point, this fraction of source points closest
    * to the current point is taken into account for computing
    * a least-squares regression.
    * A sensible value is usually 0.25 to 0.5, the default value is
    * {@link #DEFAULT\_BANDWIDTH}.
    * @param robustnessIters This many robustness iterations are done.
    * A sensible value is usually 0 (just the initial fit without any
    * robustness iterations) to 4, the default value is
    * {@link #DEFAULT\_ROBUSTNESS\_ITERS}.
    * @param accuracy If the median residual at a certain robustness iteration
    * is less than this amount, no more iterations are done.
    * @throws OutOfRangeException if bandwidth does not lie in the interval [0,1].
    * @throws NotPositiveException if {@code robustnessIters} is negative.
    * @see #LoessInterpolator\(double, int\)
    * @since 2.1
    */
    public LoessInterpolator(double bandwidth, int robustnessIters, double accuracy)
        throws OutOfRangeException,
               NotPositiveException {
        if (bandwidth < 0 ||
            bandwidth > 1) {
            throw new OutOfRangeException(LocalizedFormats.BANDWIDTH, bandwidth, 0, 1);
        }
        this.bandwidth = bandwidth;
        if (robustnessIters < 0) {
            throw new NotPositiveException(LocalizedFormats.ROBUSTNESS_ITERATIONS, robustnessIters);
        }
        this.robustnessIters = robustnessIters;
        this.accuracy = accuracy;
    }
    /**
     * Compute an interpolating function by performing a loess fit
     * on the data at the original abscissae and then building a cubic spline

```

```

    * with a
    * {@link org.apache.commons.math4.legacy.analysis.interpolation.SplineInterpolator}
    * on the resulting fit.
    *
    * @param xval the arguments for the interpolation points
    * @param yval the values for the interpolation points
    * @return A cubic spline built upon a loess fit to the data at the
    original abscissae
    * @throws NonMonotonicSequenceException if {@code xval} not sorted
    in
    * strictly increasing order.
    * @throws DimensionMismatchException if {@code xval} and {@code yval} have
    al} have
    * different sizes.
    * @throws NoDataException if {@code xval} or {@code yval} has zero
    size.
    * @throws NotFiniteNumberException if any of the arguments and values are
    ues are
    * not finite real numbers.
    * @throws NumberIsTooSmallException if the bandwidth is too small
    to
    * accommodate the size of the input data (i.e. the bandwidth must
    be
    * larger than  $2/n$ ).
    */
    @Override
    public final PolynomialSplineFunction interpolate(final double[] xval,
                                                    final double[] yval)
    al)
        throws NonMonotonicSequenceException,
               DimensionMismatchException,
               NoDataException,
               NotFiniteNumberException,
               NumberIsTooSmallException {
        return new SplineInterpolator().interpolate(xval, smooth(xval,
yval));
    }
    /**
    * Compute a weighted loess fit on the data at the original abscissae.
    *
    * @param xval Arguments for the interpolation points.

```

```

    * @param yval Values for the interpolation points.
    * @param weights point weights: coefficients by which the robustness weight
    * of a point is multiplied.
    * @return the values of the loess fit at corresponding original abscissae.
    * @throws NonMonotonicSequenceException if {@code xval} not sorted in
    * strictly increasing order.
    * @throws DimensionMismatchException if {@code xval} and {@code yval} have
    * different sizes.
    * @throws NoDataException if {@code xval} or {@code yval} has zero size.
    * @throws NotFiniteNumberException if any of the arguments and values are
    * not finite real numbers.
    * @throws NumberIsTooSmallException if the bandwidth is too small to
    * accommodate the size of the input data (i.e. the bandwidth must be
    * larger than  $2/n$ ).
    * @since 2.1
    */
    public final double[] smooth(final double[] xval, final double[] yval,
                                final double[] weights)
        throws NonMonotonicSequenceException,
               DimensionMismatchException,
               NoDataException,
               NotFiniteNumberException,
               NumberIsTooSmallException {
        if (xval.length != yval.length) {
            throw new DimensionMismatchException(xval.length, yval.length);
        }
        final int n = xval.length;
        if (n == 0) {
            throw new NoDataException();
        }
        NotFiniteNumberException.check(xval);
        NotFiniteNumberException.check(yval);
        NotFiniteNumberException.check(weights);
        MathArrays.checkOrder(xval);

```

```

        if (n == 1) {
            return new double[] { yval[0] };
        }
        if (n == 2) {
            return new double[] { yval[0], yval[1] };
        }
        int bandwidthInPoints = (int) (bandwidth * n);
        if (bandwidthInPoints < 2) {
            throw new NumberIsTooSmallException(LocalizedFormats.BANDWIDTH,
                                                    bandwidthInPoints, 2, true);
        }
        final double[] res = new double[n];
        final double[] residuals = new double[n];
        final double[] sortedResiduals = new double[n];
        final double[] robustnessWeights = new double[n];
        // Do an initial fit and 'robustnessIters' robustness iterations.
        // This is equivalent to doing 'robustnessIters+1' robustness iterations
        // starting with all robustness weights set to 1.
        Arrays.fill(robustnessWeights, 1);
        for (int iter = 0; iter <= robustnessIters; ++iter) {
            final int[] bandwidthInterval = { 0, bandwidthInPoints - 1 };
            // At each x, compute a local weighted linear regression
            for (int i = 0; i < n; ++i) {
                final double x = xval[i];
                // Find out the interval of source points on which
                // a regression is to be made.
                if (i > 0) {
                    updateBandwidthInterval(xval, weights, i, bandwidthInterval);
                }
                final int ileft = bandwidthInterval[0];
                final int iright = bandwidthInterval[1];
                // Compute the point of the bandwidth interval that is
                // farthest from x
                final int edge;
                if (xval[i] - xval[ileft] > xval[iright] - xval[i]) {
                    edge = ileft;
                } else {
                    edge = iright;
                }
            }
        }
    }

```

```

// Compute a Least-squares linear fit weighted by
// the product of robustness weights and the tricube
// weight function.
// See http://en.wikipedia.org/wiki/Linear\_regression
// (section "Univariate linear case")
// and http://en.wikipedia.org/wiki/Weighted\_Least\_squa
res
// (section "Weighted Least squares")
double sumWeights = 0;
double sumX = 0;
double sumXSquared = 0;
double sumY = 0;
double sumXY = 0;
double denom = JdkMath.abs(1.0 / (xval[edge] - x));
for (int k = ileft; k <= irect; ++k) {
    final double xk = xval[k];
    final double yk = yval[k];
    final double dist = (k < i) ? x - xk : xk - x;
    final double w = tricube(dist * denom) * robustn
essWeights[k] * weights[k];
    final double xkw = xk * w;
    sumWeights += w;
    sumX += xkw;
    sumXSquared += xk * xkw;
    sumY += yk * w;
    sumXY += yk * xkw;
}
final double meanX = sumX / sumWeights;
final double meanY = sumY / sumWeights;
final double meanXY = sumXY / sumWeights;
final double meanXSquared = sumXSquared / sumWeights;
final double beta;
if (JdkMath.sqrt(JdkMath.abs(meanXSquared - meanX * mea
nX)) < accuracy) {
    beta = 0;
} else {
    beta = (meanXY - meanX * meanY) / (meanXSquared - m
eanX * meanX);
}
final double alpha = meanY - beta * meanX;
res[i] = beta * x + alpha;
residuals[i] = JdkMath.abs(yval[i] - res[i]);
}
// No need to recompute the robustness weights at the last

```

```

        // iteration, they won't be needed anymore
        if (iter == robustnessIters) {
            break;
        }
        // Recompute the robustness weights.
        // Find the median residual.
        // An arraycopy and a sort are completely tractable here,
        // because the preceding loop is a lot more expensive
        System.arraycopy(residuals, 0, sortedResiduals, 0, n);
        Arrays.sort(sortedResiduals);
        final double medianResidual = sortedResiduals[n / 2];
        if (JdkMath.abs(medianResidual) < accuracy) {
            break;
        }
        for (int i = 0; i < n; ++i) {
            final double arg = residuals[i] / (6 * medianResidual);
            if (arg >= 1) {
                robustnessWeights[i] = 0;
            } else {
                final double w = 1 - arg * arg;
                robustnessWeights[i] = w * w;
            }
        }
    }
    return res;
}

/**
 * Compute a loess fit on the data at the original abscissae.
 *
 * @param xval the arguments for the interpolation points
 * @param yval the values for the interpolation points
 * @return values of the loess fit at corresponding original abscissae
 *
 * @throws NonMonotonicSequenceException if {@code xval} not sorted
 *         in strictly increasing order.
 * @throws DimensionMismatchException if {@code xval} and {@code yval} have
 *         different sizes.
 * @throws NoDataException if {@code xval} or {@code yval} has zero
 *         size.
 * @throws NotFiniteNumberException if any of the arguments and values are
 *         not finite real numbers.

```



```

final int[] bandwidthInterval;

interval) {
    final int left = bandwidthInterval[0];
    final int right = bandwidthInterval[1];
    // The right edge should be adjusted if the next point to the right
    // is closer to xval[i] than the leftmost point of the current interval
    int nextRight = nextNonzero(weights, right);
    int nextLeft = left;
    while (nextRight < xval.length &&
           xval[nextRight] - xval[i] < xval[i] - xval[nextLeft]) {
        nextLeft = nextNonzero(weights, bandwidthInterval[0]);
        bandwidthInterval[0] = nextLeft;
        bandwidthInterval[1] = nextRight;
        nextRight = nextNonzero(weights, nextRight);
    }
}

/**
 * Return the smallest index {@code j} such that
 * {@code j > i && (j == weights.length || weights[j] != 0)}.
 *
 * @param weights Weights array.
 * @param i Index from which to start search.
 * @return the smallest compliant index.
 */
private static int nextNonzero(final double[] weights, final int i)
{
    int j = i + 1;
    while(j < weights.length && weights[j] == 0) {
        ++j;
    }
    return j;
}

/**
 * Compute the
 * <a href="http://en.wikipedia.org/wiki/Local_regression#Weight_function">tricube</a>
 * weight function.
 *
 * @param x Argument.
 * @return <code>(1 - |x|3)3</code> for |x| < 1, 0 otherwise.
 */

```

```

        private static double tricube(final double x) {
            final double absX = JdkMath.abs(x);
            if (absX >= 1.0) {
                return 0.0;
            }
            final double tmp = 1 - absX * absX * absX;
            return tmp * tmp * tmp;
        }
    }
    //vt
    class LIoess_VT {
        /* ----- helpers ----- */
        @FunctionalInterface private interface Throwing { void run() throws
        Exception; }
        private static void headline(String name){ System.out.println("\n==
        == " + name + " ===="); }
        private static void expectThrow(String name, Throwing r, Class<?> e
        xpected){
            System.out.println("---- " + name + " ----");
            try {
                r.run();
                System.out.printf("[FAIL] %s should throw %s%n",
                    name, expected.getSimpleName());
            } catch (Throwable t){
                System.out.println("threw: " + t.getClass().getName() + " -
                " + t.getMessage());
                System.out.printf("[%s] %s threw expected%n",
                    t.getClass().equals(expected) ? "PASS" : "FAIL", na
                me);
            }
        }
        private static String fmt(double v){
            if (Double.isNaN(v)) return "NaN";
            if (v == Double.POSITIVE_INFINITY) return "+Inf";
            if (v == Double.NEGATIVE_INFINITY) return "-Inf";
            if (v == 0.0) return (Double.doubleToRawLongBits(v)>>>63)==1 ?
            "-0.0" : "0.0";
            return String.format(java.util.Locale.ROOT, "%.17g", v);
        }
        private static void checkArrayLen(String name, double[] a, int n){
            boolean ok = (a != null && a.length == n);
            System.out.printf("[%s] %s expected length=%d actual=%s%n",
                ok ? "PASS" : "FAIL", name, n, (a==null? "null" : Integer.
                toString(a.length)));
        }
    }

```

```

    }
    private static void checkRMSE(String name, double[] ref, double[] got, double tol){
        double se = 0; int n = ref.length;
        for (int i=0;i<n;i++){
            double d = ref[i] - got[i];
            se += d*d;
        }
        double rmse = Math.sqrt(se / n);
        boolean ok = rmse <= tol || Double.isNaN(tol);
        System.out.printf("[%s] %s RMSE=%s tol=%s%n",
            ok ? "PASS" : "FAIL", name, fmt(rmse), fmt(tol));
    }
    private static double[] linspace(double a, double b, int n){
        double[] x = new double[n];
        double h = (b - a) / (n - 1);
        for (int i=0;i<n;i++) x[i] = a + i*h;
        return x;
    }
    private static double[] y_of_line(double[] x, double m, double b){
        double[] y = new double[x.length];
        for (int i=0;i<x.length;i++) y[i] = m*x[i] + b;
        return y;
    }
    private static double[] y_of_sin(double[] x, double A, double w){
        double[] y = new double[x.length];
        for (int i=0;i<x.length;i++) y[i] = A*Math.sin(w*x[i]);
        return y;
    }
    public void Vtest() throws Exception {
        /* ===== Constructor parameter guards =====
        ===== */
        // VT01: bandwidth <= 0 -> OutOfRange
        expectThrow("VT01 ctor bandwidth<=0",
            () -> new LoessInterpolator(0.0, 2, 1e-12),
            OutOfRangeException.class);
        // VT02: bandwidth > 1 -> OutOfRange
        expectThrow("VT02 ctor bandwidth>1",
            () -> new LoessInterpolator(1.0000001, 2, 1e-12),
            OutOfRangeException.class);
        // VT03: robustnessIters < 0 -> NotPositive
        expectThrow("VT03 ctor robustnessIters<0",
            () -> new LoessInterpolator(0.3, -1, 1e-12),
            NotPositiveException.class);
    }

```

```

// VT04: accuracy <= 0 -> NotStrictlyPositive
expectThrow("VT04 ctor accuracy<=0",
    () -> new LoessInterpolator(0.3, 2, 0.0),
    NotStrictlyPositiveException.class);
// VT05: ctor OK
headline("VT05 ctor OK");
LoessInterpolator L_ok = new LoessInterpolator(0.3, 2, 1e-12);
System.out.println("[PASS] constructed with (bandwidth=0.3, ite
rs=2, acc=1e-12)");
/* ===== smooth(x,y) guards =====
== */
// VT06: x=null -> NullArgument
expectThrow("VT06 x=null",
    () -> L_ok.smooth(null, new double[]{1,2}),
    NullArgumentException.class);
// VT07: y=null -> NullArgument
expectThrow("VT07 y=null",
    () -> L_ok.smooth(new double[]{0,1}, null),
    NullArgumentException.class);
// VT08: |x|=0 or |y|=0 -> NoData
expectThrow("VT08 |x|=0",
    () -> L_ok.smooth(new double[] {}, new double[] {}),
    NoDataException.class);
// VT09: |x|≠|y| -> DimensionMismatch
expectThrow("VT09 length mismatch",
    () -> L_ok.smooth(new double[]{0,1,2,3,4}, new double[]
{1,2,3,4}),
    DimensionMismatchException.class);
// VT10: |x|<2 -> NumberIsTooSmall
expectThrow("VT10 |x|<2",
    () -> L_ok.smooth(new double[]{0.0}, new double[]{1.0})
,
    NumberIsTooSmallException.class);
// VT11:
expectThrow("VT11 x has NaN",
    () -> L_ok.smooth(new double[]{0.0, Double.NaN, 2.0}, n
ew double[]{1.0, 2.0, 3.0}),
    NotFiniteNumberException.class);
// VT12:
expectThrow("VT12 y has +Inf",
    () -> L_ok.smooth(new double[]{0,1,2}, new double[]{1,
Double.POSITIVE_INFINITY, 3}),
    NotFiniteNumberException.class);
// VT13:

```

```

    expectThrow("VT13 x non-strict (duplicate)",
        () -> L_ok.smooth(new double[]{0, 1, 1, 2}, new double[
] {3, 4, 5, 6})),
        NonMonotonicSequenceException.class);
// VT14:
    expectThrow("VT14 x non-strict (descending)",
        () -> L_ok.smooth(new double[]{2, 1, 0}, new double[] {6
, 4, 3})),
        NonMonotonicSequenceException.class);
/* ===== success paths ===== */
    LoessInterpolator L_def = new LoessInterpolator();
// VT15:
    headline("VT15 linear data recovery");
    {
        int n = 101;
        double[] x = linspace(-5, 5, n);
        double[] y = y_of_line(x, 2.0, -1.0); //  $y = 2x - 1$ 
        double[] s = L_def.smooth(x, y);
        checkArrayLen("VT15 length", s, n);
        checkRMSE("VT15 rmse", y, s, 1e-10);
    }
// VT16:
    headline("VT16 sinusoid smoothing");
    {
        int n = 201;
        double[] x = linspace(0, 2*Math.PI, n);
        double[] y = y_of_sin(x, 1.0, 1.0); //  $\sin(x)$ 
        double[] s = L_ok.smooth(x, y);
        checkArrayLen("VT16 length", s, n);
        checkRMSE("VT16 rmse", y, s, 1e-2);
    }
// VT17:
    headline("VT17 noisy linear denoise");
    {
        java.util.Random R = new java.util.Random(20251028L);
        int n = 301;
        double[] x = linspace(-3, 3, n);
        double[] yClean = y_of_line(x, -0.7, 2.0);
        double[] yNoisy = yClean.clone();
        for (int i=0; i<n; i++) yNoisy[i] += 0.05*R.nextGaussian();
        double[] s = L_ok.smooth(x, yNoisy);
        double seNoisy=0, seSm=0;
        for (int i=0; i<n; i++){
            double eN = yClean[i]-yNoisy[i];

```

```

        double eS = yClean[i]-s[i];
        seNoisy += eN*eN;
        seSm += eS*eS;
    }
    double rmseNoisy = Math.sqrt(seNoisy/n);
    double rmseSm = Math.sqrt(seSm/n);
    boolean ok = rmseSm < rmseNoisy;
    System.out.printf("[%s] VT17 rmse noisy=%s smoothed=%s%n",
        ok ? "PASS" : "FAIL", fmt(rmseNoisy), fmt(rmseSm));
}
// VT18:
headline("VT18 tiny spacing & wide dynamic range");
{
    int n = 20;
    double[] x = new double[n];
    for (int i=0;i<n;i++) x[i] = i * 1e-12;
    double[] y = new double[n];
    for (int i=0;i<n;i++) y[i] = 1e150 + i*1e138;
    double[] s = L_ok.smooth(x, y);
    checkArrayLen("VT18 length", s, n);
    boolean finite = true;
    for (double v: s) finite &= !(Double.isNaN(v) || Double.isI
nfinite(v));
    System.out.printf("[%s] VT18 all finite%n", finite ? "PASS"
: "FAIL");
}
// VT19:
headline("VT19 minimal n=2");
{
    double[] x = new double[]{0.0, 1.0};
    double[] y = new double[]{2.0, 4.0};
    double[] s = L_ok.smooth(x, y);
    checkArrayLen("VT19 length", s, 2);
    checkRMSE("VT19 rmse", y, s, 1e-12);
}
// VT20:
headline("VT20 constant series");
{
    int n = 50;
    double[] x = linspace(0, 1, n);
    double[] y = new double[n];
    Arrays.fill(y, 7.5);
    double[] s = L_def.smooth(x, y);
    checkArrayLen("VT20 length", s, n);
}

```

```

        checkRMSE("VT20 rmse", y, s, 1e-12);
    }
    System.out.println("\n[Done] LoessInterpolator VT01-
VT20 executed.");
}
}
//ft
class LOESS_FT {
    private static void headline(String h){ System.out.println("\n====
" + h + " ===="); }
    private static String fmtD(double v){
        if (Double.isNaN(v)) return "NaN";
        if (v == Double.POSITIVE_INFINITY) return "+Inf";
        if (v == Double.NEGATIVE_INFINITY) return "-Inf";
        if (v == 0.0) return (Double.doubleToRawLongBits(v)>>>63)==1 ?
"-0.0" : "0.0";
        return String.format(java.util.Locale.ROOT, "%.7g", v);
    }
    private static void reportOK(int idx, double[] out){
        System.out.printf("FT%02d -> OK (out.length=%d)%n", idx, out==n
ull?0:out.length);
    }
    private static void reportEx(int idx, Throwable t){
        System.out.printf("FT%02d -> EX (%s): %s%n", idx, t.getClass().
getSimpleName(), t.getMessage());
    }
    // random monotone-ish x generator (may be non-
monotone if step can be negative)
    private static double[] genX(Random r, int n){
        double[] a = new double[n];
        double cur = r.nextDouble()*10.0 - 5.0;
        for (int i=0;i<n;i++){
            // allow negative step sometimes to introduce non-
monotonicity
            double step = (r.nextDouble() < 0.85) ? Math.abs(r.nextGaus
sian()*2.0 + 0.5) : -Math.abs(r.nextGaussian()*2.0 + 0.5);
            cur += step;
            // sometimes inject extreme values
            if (r.nextDouble() < 0.02) a[i] = Double.NaN;
            else if (r.nextDouble() < 0.02) a[i] = Double.POSITIVE_INFI
NITY;
            else if (r.nextDouble() < 0.02) a[i] = Double.NEGATIVE_INFI
NITY;
            else a[i] = cur;

```

```

    }
    return a;
}

private static double[] genY(Random r, int n){
    double[] a = new double[n];
    for (int i=0;i<n;i++){
        if (r.nextDouble() < 0.03) a[i] = Double.NaN;
        else if (r.nextDouble() < 0.02) a[i] = Double.POSITIVE_INFINITY;
        else if (r.nextDouble() < 0.02) a[i] = Double.NEGATIVE_INFINITY;
        else a[i] = r.nextGaussian() * Math.pow(10, r.nextInt(10)-5);
    }
    return a;
}

// randomly insert duplicates / shuffle a little
private static void perturbX(Random r, double[] x){
    if (x.length < 2) return;
    if (r.nextDouble() < 0.15) { // duplicate neighbor
        int i = r.nextInt(x.length-1);
        x[i+1] = x[i];
    }
    if (r.nextDouble() < 0.10) { // swap two
        int i = r.nextInt(x.length), j = r.nextInt(x.length);
        double t = x[i]; x[i] = x[j]; x[j] = t;
    }
}

public void Ftest() {
    final Random R = new Random(20251028L);
    for (int id = 1; id <= 20; id++) {
        headline(String.format("FT%02d", id));
        try {
            // random constructor params (some intentionally out of
            // valid range)
            double bandwidth = (R.nextDouble() < 0.9) ? (0.01 + R.nextDouble()*0.99) : (-1.0 + R.nextDouble()*3.0); // mostly within (0,1], sometimes outside
            int robustness = (R.nextDouble() < 0.9) ? Math.abs(R.nextInt()%5) : -Math.abs(R.nextInt()%5); // mostly >=0 sometimes negative
            double accuracy = (R.nextDouble() < 0.95) ? Math.pow(10, -6 - R.nextInt(6)) : (R.nextDouble()*0.0); // mostly >0 sometimes 0
            // choose length randomly 0..40 (allow zero length)
            int n = R.nextInt(41);

```

```

        // sometimes force very small n (0 or 1)
        if (R.nextDouble() < 0.08) n = R.nextInt(2);
        double[] x = genX(R, Math.max(0, n));
        double[] y = genY(R, Math.max(0, n));
        // random perturbations to induce edge-cases
        perturbX(R, x);
        if (R.nextDouble() < 0.05 && x.length>0) x[R.nextInt(x.
length)] = Double.NaN;
        if (R.nextDouble() < 0.05 && y.length>0) y[R.nextInt(y.
length)] = Double.POSITIVE_INFINITY;
        // 50% chance use default ctor; otherwise use random pa
rams
        LoessInterpolator LI;
        if (R.nextDouble() < 0.5) {
            LI = new LoessInterpolator();
            System.out.printf("FT%02d ctor=default n=%d bandwi
dth=default%n", id, n);
        } else {
            LI = new LoessInterpolator(bandwidth, robustness, a
ccuracy);
            System.out.printf("FT%02d ctor=(b=%.5g,it=%d,acc=%s
) n=%d%n", id, bandwidth, robustness, fmtD(accuracy), n);
        }
        // invoke smooth
        double[] out = LI.smooth(x, y);
        // report success: show first/last/len
        System.out.printf("FT%02d -> OK out[0]=%s out[last]=%s
len=%d%n",
                        id,
                        out.length==0? "EMPTY" : fmtD(out[0]),
                        out.length==0? "EMPTY" : fmtD(out[out.length-
1]),
                        out.length);
    } catch (Throwable t) {
        reportEx(id, t);
    }
}
System.out.println("\n[Done] LOESS FT01-
FT20 fuzzing executed.");
}
}
//Z3
class LOESS_Z3 {
    /* ----- helpers ----- */

```

```

    @FunctionalInterface private interface Throwing { void run() throws
Exception; }
    private static void headline(String name){ System.out.println("\n==
== " + name + " ==="); }
    private static void expectThrow(String name, Throwing r, Class<?> e
xpected){
        System.out.println("---- " + name + " ----");
        try { r.run();
            System.out.printf("[FAIL] %s should throw %s\n", name, expe
cted.getSimpleName());
        } catch (Throwable t){
            System.out.println("threw: " + t.getClass().getName() + " -
" + t.getMessage());
            System.out.printf("[%s] %s threw expected\n",
                t.getClass().equals(expected) ? "PASS" : "FAIL", na
me);
        }
    }
    private static String fmt(double v){
        if (Double.isNaN(v)) return "NaN";
        if (v == Double.POSITIVE_INFINITY) return "+Inf";
        if (v == Double.NEGATIVE_INFINITY) return "-Inf";
        if (v == 0.0) return (Double.doubleToRawLongBits(v)>>>63)==1 ?
"-0.0" : "0.0";
        return String.format(java.util.Locale.ROOT, "%.17g", v);
    }
    private static double[] linspace(double a, double b, int n){
        double[] x = new double[n];
        double h = (b - a) / (n - 1);
        for (int i=0;i<n;i++) x[i] = a + i*h;
        return x;
    }
    private static double[] y_of_line(double[] x, double m, double b){
        double[] y = new double[x.length];
        for (int i=0;i<x.length;i++) y[i] = m*x[i] + b;
        return y;
    }
    private static void checkLen(String name, double[] arr, int n){
        boolean ok = (arr != null && arr.length == n);
        System.out.printf("[%s] %s expectedLen=%d actual=%s\n",
            ok ? "PASS" : "FAIL", name, n, (arr==null? "null" : Int
eger.toString(arr.length)));
    }
}

```

```

    private static void checkRMSE(String name, double[] ref, double[] got, double tol){
        double se = 0; int n = ref.length;
        for (int i=0;i<n;i++){ double d = ref[i]-got[i]; se += d*d; }
        double rmse = Math.sqrt(se/n);
        boolean ok = rmse <= tol;
        System.out.printf("[%s] %s RMSE=%s tol=%s%n",
            ok ? "PASS" : "FAIL", name, fmt(rmse), fmt(tol));
    }

    public void Z3test() throws Exception {
        // Z301: x=null or y=null → NullArgumentException
        expectThrow("Z301 x=null → NullArgument",
            () -> new LoessInterpolator().smooth(null, new double[]
{1.0})),
            NullArgumentException.class);
        // Z302: |x|=0 or |y|=0 → NoDataException
        expectThrow("Z302 |x|=0 → NoData",
            () -> new LoessInterpolator().smooth(new double[] {}, ne
w double[] {1,2,3,4,5})),
            NoDataException.class);
        // Z303: |x|≠|y| → DimensionMismatchException
        expectThrow("Z303 |x|=5, |y|=4 → DimensionMismatch",
            () -> new LoessInterpolator().smooth(
new double[] {0,1,2,3,4}, new double[] {10,11,12,
13})),
            DimensionMismatchException.class);
        // Z304: |x|<2 → NumberIsTooSmallException
        expectThrow("Z304 |x|<2 (|x|=1)",
            () -> new LoessInterpolator().smooth(new double[] {0.0},
new double[] {1.0})),
            NumberIsTooSmallException.class);
        // Z305: ∃ non-finite in x/y → NotFiniteNumberException
        expectThrow("Z305 y has NaN → NotFinite",
            () -> new LoessInterpolator().smooth(
new double[] {0.0, 1.0, 2.0}, new double[] {5.0,
Double.NaN, 9.0})),
            NotFiniteNumberException.class);
        // Z306: x not strictly increasing → NonMonotonicSequenceExcept
ion
        expectThrow("Z306 x non-strict (0,1,1,3)",
            () -> new LoessInterpolator().smooth(
new double[] {0.0, 1.0, 1.0, 3.0}, new double[] {
0.0, 1.0, 2.0, 3.0})),
            NonMonotonicSequenceException.class);
    }

```

```

// Z307: success: valid monotone & finite,  $|x|=|y|\geq 2 \rightarrow$  None
headline("Z307 success path (default ctor; n big enough for bandwidth)");
{
    int n = 20;
    double[] x = linspace(0, 5, n);
    double[] y = y_of_line(x, 2.0, 1.0); //  $y = 2x + 1$ 
    double[] s = new LoessInterpolator().smooth(x, y);
    checkLen("Z307 length", s, x.length);
    checkRMSE("Z307 linear recovery", y, s, 1e-10);
}
// Z308: NoData variant:  $|y|=0$  ( $|x|>0$ )  $\rightarrow$  NoDataException
expectThrow("Z308  $|y|=0$  ( $|x|>0$ )",
    () -> new LoessInterpolator().smooth(new double[]{0.0, 1.0}, new double[]{}),
    NoDataException.class);
// Z309: DimensionMismatch variant:  $|x|=3$ ,  $|y|=2 \rightarrow$  DimensionMismatchException
expectThrow("Z309  $|x|=3$ ,  $|y|=2$ ",
    () -> new LoessInterpolator().smooth(new double[]{-1.0, 0.0, 2.0},
        new double[]{4.0, 8.0}),
    DimensionMismatchException.class);
// Z310: NonMonotonic edge (equal neighbors)  $\rightarrow$  NonMonotonicSequenceException
expectThrow("Z310 equal neighbors (10,10)",
    () -> new LoessInterpolator().smooth(new double[]{10.0, 10.0},
        new double[]{-2.0, -2.0}),
    NonMonotonicSequenceException.class);
// Z311:
expectThrow("Z311 y=null  $\rightarrow$  NullArgument",
    () -> new LoessInterpolator().smooth(new double[]{0.0}, null),
    NullArgumentException.class);
// Z312:
expectThrow("Z312  $|y|=0$  with  $|x|=1 \rightarrow$  NoData",
    () -> new LoessInterpolator().smooth(new double[]{7.0}, new double[]{}),
    NoDataException.class);
// Z313:
expectThrow("Z313  $|x|=1$  again",

```

```

        () -> new LoessInterpolator().smooth(new double[]{2.5},
new double[]{-1.5}),
        NumberIsTooSmallException.class);
// Z314:
expectThrow("Z314 x has +Inf",
        () -> new LoessInterpolator().smooth(
            new double[]{0.0, Double.POSITIVE_INFINITY, 2.0
},
            new double[]{1.0, 2.0, 3.0}),
        NotFiniteNumberException.class);
// Z315:
expectThrow("Z315 y has -Inf",
        () -> new LoessInterpolator().smooth(
            new double[]{0.0, 1.0, 2.0},
            new double[]{1.0, Double.NEGATIVE_INFINITY, 3.0
}),
        NotFiniteNumberException.class);
// Z316:
expectThrow("Z316 x strictly descending",
        () -> new LoessInterpolator().smooth(
            new double[]{3.0, 2.0, 1.0, 0.0},
            new double[]{0.0, 1.0, 2.0, 3.0}),
        NonMonotonicSequenceException.class);
// Z317:
expectThrow("Z317 ctor bandwidth<=0",
        () -> { new LoessInterpolator(0.0, 2, 1e-12); },
        OutOfRangeException.class);
// Z318:
expectThrow("Z318 ctor bandwidth>1",
        () -> { new LoessInterpolator(1.1, 1, 1e-12); },
        OutOfRangeException.class);
// Z319:
expectThrow("Z319a ctor iters<0",
        () -> { new LoessInterpolator(0.3, -1, 1e-12); },
        NotPositiveException.class);
expectThrow("Z319b ctor accuracy<=0",
        () -> { new LoessInterpolator(0.3, 1, 0.0); },
        NotStrictlyPositiveException.class);
// Z320
headline("Z320 success with custom ctor");
{
    LoessInterpolator Lc = new LoessInterpolator(0.4, 2, 1e-
12);
    double[] x = linspace(-3, 3, 121);

```

```

        double[] y = y_of_line(x, -0.7, 2.0); // y = -0.7x + 2
        double[] s = Lc.smooth(x, y);
        checkLen("Z320 length", s, x.length);
        int mid = x.length/2;
        double rmseTol = 5e-11;
        checkRMSE("Z320 linear recovery", y, s, rmseTol);
        System.out.printf("Z320 samples: s[0]=%s s[mid]=%s s[last]=%s\n",
            fmt(s[0]), fmt(s[mid]), fmt(s[s.length-1]));
    }
    System.out.println("\n[Done] LoessInterpolator Z301-Z320 executed.");
}
}
/* ----- Launcher ----- */
final class LIoess_Runner {
    public static void main(String[] args) throws Exception {
        new LIoess_VT().Vtest();
//        new LOESS_FT().Ftest();
//        new LOESS_Z3().Z3test();
    }
}

```

- Coverage report

1. Test case number: 20
2. V-Method testing branch coverage: 90%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
LoessInterpolator.java	43.8 %	85	109	194
> LIoess_VT	74.1 %	40	14	54
> LoessInterpolator	90.0 %	45	5	50
nextNonzero(double[], int)	75.0 %	3	1	4
LoessInterpolator(double, int, double)	83.3 %	5	1	6
smooth(double[], double[], double[])	90.6 %	29	3	32
tricube(double)	100.0 %	2	0	2
updateBandwidthInterval(double[], c)	100.0 %	4	0	4
smooth(double[], double[])	100.0 %	2	0	2
LoessInterpolator()		0	0	0
> LIoess_Runner		0	0	0

3. V-Method testing instruction coverage: 93.8%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
LoessInterpolator.java	42.9 %	1,295	1,726	3,021
> LIoess_Runner	62.5 %	5	3	8
> LIoess_VT	75.9 %	758	241	999
> LoessInterpolator	93.8 %	532	35	567
nextNonzero(double[], int)	94.4 %	17	1	18
smooth(double[], double[], double[])	95.2 %	360	18	378
tricube(double)	100.0 %	23	0	23
updateBandwidthInterval(double[], c)	100.0 %	54	0	54
LoessInterpolator()	100.0 %	12	0	12
LoessInterpolator(double, int, double)	100.0 %	40	0	40
smooth(double[], double[])	100.0 %	26	0	26

4. Fuzz testing branch coverage: 22%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
LoessInterpolator.java	31.4 %	61	133	194
LoessInterpolator	22.0 %	11	39	50
smooth(double[], double[], double[])	21.9 %	7	25	32
LoessInterpolator(double, int, double)	50.0 %	3	3	6
smooth(double[], double[])	50.0 %	1	1	2
LoessInterpolator()		0	0	0
LOESS_FT	83.3 %	50	10	60
Lloess_Runner		0	0	0

5. Fuzz testing instruction coverage: 19.2%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
LoessInterpolator.java	20.9 %	631	2,390	3,021
LoessInterpolator	19.2 %	109	458	567
smooth(double[], double[], double[])	15.1 %	57	321	378
LoessInterpolator(double, int, double)	55.0 %	22	18	40
smooth(double[], double[])	69.2 %	18	8	26
LoessInterpolator()	100.0 %	12	0	12
Lloess_Runner	62.5 %	5	3	8
LOESS_FT	93.8 %	517	34	551

6. Z3 branch coverage: 72%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
LoessInterpolator.java	27.8 %	54	140	194
LOESS_Z3	60.0 %	18	12	30
LoessInterpolator	72.0 %	36	14	50
smooth(double[], double[], double[])	62.5 %	20	12	32
nextNonzero(double[], int)	75.0 %	3	1	4
LoessInterpolator(double, int, double)	83.3 %	5	1	6
tricube(double)	100.0 %	2	0	2
updateBandwidthInterval(double[], c)	100.0 %	4	0	4
smooth(double[], double[])	100.0 %	2	0	2
LoessInterpolator()		0	0	0
Lloess_Runner		0	0	0

7. Z3 instruction coverage: 83.2%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
LoessInterpolator.java	41.0 %	1,239	1,782	3,021
Lloess_Runner	62.5 %	5	3	8
LoessInterpolator	83.2 %	472	95	567
smooth(double[], double[], double[])	79.4 %	300	78	378
nextNonzero(double[], int)	94.4 %	17	1	18
tricube(double)	100.0 %	23	0	23
updateBandwidthInterval(double[], c)	100.0 %	54	0	54
LoessInterpolator()	100.0 %	12	0	12
LoessInterpolator(double, int, double)	100.0 %	40	0	40
smooth(double[], double[])	100.0 %	26	0	26
LOESS_Z3	85.0 %	762	134	896